

TaxaMatch

Store and standardize biological match data for the TaxaID ecosystem. Ingests raw match results from external classification tools and produces a canonical match object for input to TaxaLikely (likelihood conversion) or TaxaAssign (direct assignment).

The main input to TaxaID is a table where each row pairs an observation (sequence, image, or sound recording) with a candidate taxon from a reference database and a match score indicating their similarity. TaxaMatch standardizes column names and taxonomy so that downstream packages can interpret this table. Standardization is a prerequisite for calibrating scores into likelihoods – without it, raw scores may be mistaken for probabilities, leading to overconfident or inconsistent assignments.

TaxaMatch can also produce match tables from raw sequence data via NCBI (National Center for Biotechnology Information, U.S. National Library of Medicine, National Institutes of Health, Bethesda, Maryland) BLAST (Basic Local Alignment Search Tool; Altschul et al. 1990), though most users will start from an existing bioinformatics pipeline.

Supported Data Types

Data type	Input format	Function
DNA sequences	DADA2 seqtab, FASTA, DNASTringSet	<code>read_sequence_table()</code>
BLAST results	Remote NCBI or local rBLAST	<code>blast_sequences()</code>
Images	Animl CSV export	<code>read_animl_output()</code>
Images (iNat CV)	iNaturalist CV API (live submission)	<code>score_image_inat()</code>
Acoustics	BirdNET-Analyzer CSV	<code>read_birdnet_output()</code>

Installation

```
# Requires TaxaTools (foundation package)
devtools::install("path/to/TaxaTools")
devtools::install("path/to/TaxaMatch")
```

Quick Start

```
library(TaxaMatch)

# 1. Read DADA2 output
seqs <- read_sequence_table("seqtab_nochim.rds")

# 2. Filter by length and abundance
filtered <- filter_sequences(seqs, min_length = 100, max_length = 300,
                             min_abundance = 10)

# 3. BLAST against NCBI
blast_hits <- blast_sequences(filtered, database = "nt",
                              barcode_term = "12S",
```

```

min_score = 80)

# 4. Standardize to canonical match object
match_df <- standardize_match_data(blast_hits)

# 5. Remove redundant higher-rank hypotheses
match_df <- filter_redundant_hypotheses(match_df)
# Result: one row per observation_id x taxon hypothesis, ready for TaxaLikely

```

Key Functions

- `read_sequence_table()` – ingest DADA2 seqtab matrix, FASTA, or DNASTringSet
- `filter_sequences()` – filter ASVs by length range and minimum abundance
- `blast_sequences()` – remote NCBI BLAST or local rBLAST with score window filtering and taxonomy resolution
- `read_birdnet_output()` – ingest BirdNET-Analyzer CSV files; `observation_id` encodes recording + time window ("`{stem}_{start_s}-{end_s}`"); `score` is BirdNET confidence (0–1)
- `read_animl_output()` – ingest Animl (MegaDetector + SpeciesNet) camera trap CSV; `observation_id` = image filename stem; configurable column names; supports long and wide (`n_candidates`) formats
- `standardize_match_data()` – canonical column names, taxonomy derivation via `TaxaTools::create_taxon_names()`
- `filter_redundant_hypotheses()` – drop coarser-rank rows superseded by finer-rank rows within the same lineage and observation
- `report_match()` – summarize matching for `assemble_report()`
- `corroborate_references_locally()` – free, zero-NCBI-cost first-pass reference check using your own reference set; see Reference Accession Quality
- `evaluate_reference_accessions()` – BLAST-based reference-accession quality screen; see Reference Accession Quality
- `score_reference_labels()` – combines the local and BLAST checks into one `reference_action` verdict per accession
- `flag_incongruent_references()` / `remove_incongruent_references()` – annotate or remove candidates flagged by `evaluate_reference_accessions()`
- `review_flagged_accessions()` – LLM second-look review of flagged/borderline reference accessions

Acoustic Workflow

BirdNET-Analyzer is a free tool, written in Python (Python Software Foundation, Wilmington, Delaware), from the Cornell Lab of Ornithology (Cornell University, Ithaca, New York) that classifies bird vocalizations in audio files.

Install BirdNET-Analyzer (requires Python 3.9+, ~100 MB model download):

```
pip3 install birdnetlib
```

Analyze audio files and read results into R:

```

library(TaxaMatch)

# Run BirdNET-Analyzer on a directory of audio files
script <- '
from birdnetlib import Recording
from birdnetlib.analyzer import Analyzer

```

```

import os, csv

os.makedirs("birdnet_results", exist_ok=True)
analyzer = Analyzer()
for fname in os.listdir("reference_audio/"):
    if not fname.endswith(".mp3"):
        continue
    rec = Recording(analyzer, os.path.join("reference_audio/", fname),
                    lat=37.5, lon=-122.0, min_conf=0.1)
    rec.analyze()
    out = os.path.join("birdnet_results", fname.replace(".mp3", ".BirdNET.results.csv"))
    with open(out, "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["Start (s)", "End (s)", "Scientific name", "Common name", "Confidence"])
        for d in rec.detections:
            w.writerow([d["start_time"], d["end_time"],
                        d["scientific_name"], d["common_name"], d["confidence"]])
    ,

writeLines(script, "/tmp/run_birdnet.py")
system("python3 /tmp/run_birdnet.py")

# Read results into match object format
match_df <- read_birdnet_output("birdnet_results/",
                                min_confidence = 0.1,
                                top_n          = 3L)

# Standardize and pass to TaxaLikely
match_df <- standardize_match_data(match_df,
                                    observation_id_col = "observation_id",
                                    score_col          = "score")

```

Expected BirdNET output format: one CSV per audio file, named `recording.BirdNET.results.csv`, with columns `Start (s)`, `End (s)`, `Scientific name`, `Common name`, `Confidence`. The BirdNET-Analyzer default produces up to 3 detections per 3-second window. Use `top_n = 1` to keep only the best candidate per window.

Reference training workflow: Download ground-truth recordings from Xeno-canto (Xeno-canto Foundation, Netherlands, with support from Naturalis Biodiversity Center, Leiden; <https://xeno-canto.org/>) with `TaxaLikely::fetch_xc_recording_locations()`, run BirdNET-Analyzer on the downloaded audio, then join detections back to the known species via `source_file` to label H1/H2/H3 training examples.

Camera Trap Image Workflow

`animl` (Swanson and Tobler; Conservation Technology Lab, San Diego Zoo Wildlife Alliance, San Diego, California) is an R package on CRAN that wraps MegaDetector (Microsoft AI for Earth; Microsoft Corporation, Redmond, Washington) and SpeciesNet (Google LLC, Mountain View, California) to classify camera trap images – an approach with a long track record in ecology (Tabak et al. 2019). Install it with `install.packages("animl")` (requires Python ≥ 3.12 via `reticulate`).

Read Animl results into match object format:

```
library(TaxaMatch)
```

```

# Long-format Animl CSV (one row per image x candidate species)
match_df <- read_animl_output(
  "animl_results/",          # directory of Animl export CSVs
  min_confidence = 0.5       # drop low-confidence detections
) |>
  # Remove non-wildlife detections before standardizing
  subset(!species %in% c("empty", "human", "vehicle"))

# Wide-format output (pred1/score1, pred2/score2, pred3/score3 per row)
match_df_wide <- read_animl_output(
  "animl_results/wide.csv",
  species_col = "pred",
  score_col   = "score",
  n_candidates = 3L
)

# Standardize and pass to TaxaLikely
match_df <- standardize_match_data(match_df,
                                   observation_id_col = "observation_id",
                                   score_col          = "score")

```

Expected Animl output format (long): one or more CSV files with columns `FileName`, `prediction`, `confidence`. `FileName` is the image path; `prediction` is the species label (scientific name, "empty", "human", or "vehicle"); `confidence` is the classifier posterior (0–1). Column names are configurable via `file_col`, `species_col`, and `score_col`.

Coverage audit: Check which expected species are absent from the classifier’s known list with `TaxaLikely::audit_acoustic_coverage(plausible_species, reference_species)`. Pass the result’s `$unreferenced` vector directly to `TaxaAssign::assign_taxa_llm()` as `unreferenced_taxa` (the LLM-shortcut pipeline’s own parameter for exactly this shape).

Confirmed vs. Unconfirmed Assignments

Animl output typically contains two classes of predictions: confident species-level assignments and lower-confidence results where SpeciesNet falls back to a coarse “Animal” label. TaxaID handles both:

- **Confirmed species** proceed directly through the likelihood pipeline
- **“Animal” / unconfirmed** can be re-scored with `score_image_inat()` (iNaturalist CV API), flagged for manual review, or carried forward as prior-only assignments

MegaDetector

MegaDetector (Microsoft AI for Earth) detects *whether* an image contains an animal, human, or vehicle — it does not classify to species. Animl wraps MegaDetector as its first stage to remove empty frames before species classification. If you run MegaDetector standalone (without Animl), filter its output to `detection_conf >= threshold` before passing images to a species classifier; TaxaID does not read MegaDetector output directly.

Coverage column: `read_animl_output()` can retain the bounding box area (width × height, normalised 0–1) as a `coverage` column when `bbox_cols` are specified. Smaller bounding boxes indicate a partially visible or distant animal — lower evidence per detection, directly analogous to DNA alignment coverage and acoustic recording quality. Apply a minimum-coverage threshold directly against this column before model training (e.g. `df[df$coverage >= threshold, ]`) if desired, but be cautious: hard exclusion on an

imperfect quality proxy risks systematically dropping whole taxa rather than just trimming noise. Check whether the excluded records cluster in particular species before trusting the result.

Other Image Classifiers

The match object format is classifier-agnostic. Three dedicated reader functions are available; any other tool that returns a species label and a confidence score per image can be adapted manually.

iNaturalist computer vision (direct submission) — iNaturalist (a joint initiative of the California Academy of Sciences and the National Geographic Society, San Francisco, California). `score_image_inat()` submits image files directly to the iNaturalist CV API and returns a match object in one step. Supply a directory, file vector, or single image. If images have GPS EXIF metadata, location is read automatically; otherwise supply `lat`, `lng`, and `observed_on`. Scores are in iNaturalist's 0–100 softmax scale (do not rescale). Requires an iNaturalist API token (`INAT_API_TOKEN` environment variable; free account).

```
match_df <- score_image_inat(
  "photos/",
  lat = 34.10, lng = -119.07, observed_on = "2024-09-15"
)
```

iNaturalist computer vision (saved JSON) — returns a ranked list of up to 10 candidate taxa (species, genus, or family) with softmax confidence scores (0–1) via the public API (<https://api.inaturalist.org/v2/>). Its 108,000+ taxon training set covers general wildlife globally. Use `read_inaturalist_cv_output()` on saved JSON response files:

```
# Save API response as JSON (one file per image), then:
inat_df <- read_inaturalist_cv_output(
  "inat_results/",          # directory of per-image JSON files
  score_type = "combined_score",
  min_confidence = 0.05,
  top_n = 5L
) |> subset(taxon_rank == "species")
```

SpeciesNet (Google, `google/cameratrapai`) processes camera trap images with an EfficientNetV2-M classifier + MegaDetector ensemble, covering 2,000+ labels spanning any taxonomic rank (species down to class) plus non-animal categories. Use `read_speciesnet_output()` on the batch predictions JSON:

```
# Run SpeciesNet: python -m speciesnet.scripts.run_model \
#   --folders images/ --predictions_json speciesnet_predictions.json
sn_df <- read_speciesnet_output(
  "speciesnet_predictions.json",
  min_confidence = 0.3
) |> subset(!is.na(taxon_rank))
```

InsectNet (Chiranjeevi et al. 2025; Iowa State University, Ames, Iowa; <https://insectapp.las.iastate.edu>) targets insects (2,526 species, 17 orders) with 96.4% top-1 accuracy. Unlike the classifiers above, it returns *conformal prediction sets* rather than a single ranked-confidence list — a set of species guaranteed to contain the true species with 97.5% probability. It also flags out-of-distribution images with an energy-based OOD score. The conformal output format is not currently compatible with the score-based likelihood model in TaxaLikely without access to the underlying softmax scores. A web interface is available; programmatic access to model weights is described in the paper but no public API exists at time of writing.

Classifier	Reader function	Score type	R package
Animl / SpeciesNet	<code>read_animl_output()</code>	Confidence 0–1	<code>animl</code> (CRAN)
iNaturalist CV (direct)	<code>score_image_inat()</code>	Softmax 0–100	Free API (token required)
iNaturalist CV (saved JSON)	<code>read_inaturalist_cv_output()</code>	Softmax 0–1	<code>rinat</code> (indirect)
SpeciesNet	<code>read_speciesnet_output()</code>	Confidence 0–1	Python <code>speciesnet</code>
(google/cameratrapai)			
InsectNet	<i>(not yet compatible)</i>	Conformal sets	Web app only

Reference Accession Quality

A mislabeled reference in the underlying NCBI database produces confident wrong assignments that propagate to every query matching it. TaxaMatch screens the actual candidate accessions your queries match against, before they're trusted as candidates or used to train a likelihood model, independent of whatever else happens to be in a caller's own taxon list. This applies equally to a *training* reference set (screen it before `TaxaLikely::build_sequence_matrix()/train_likelihood_model()`, which have no built-in reference-quality screening of their own) and to the accessions your *query* candidates actually matched.

The recommended screen: a free check first, then BLAST only where needed

`corroborate_references_locally()` is a **free, zero-NCBI-cost** first pass: does an independent conspecific (a different submission batch) already in your own reference set corroborate each accession's label at high identity over real sequence overlap? Run this before BLASTing anything, `evaluate_reference_accessions()`'s `skip_locally_corroborated = TRUE` (the default) skips every accession this step already resolves, so only the genuinely ambiguous remainder ever costs a BLAST call.

```
# 1. Free, zero-NCBI-cost: does an independent conspecific already in your
#    own reference set corroborate each accession's label?
local_corr <- corroborate_references_locally(seq_matrix, reference_df)
table(local_corr$local_tier) # singleton / same_batch_only / disagree / corroborated

# 2. BLAST against a broad, independent database -- only for accessions the
#    free check couldn't resolve.
ref_eval <- evaluate_reference_accessions(
  unique(reference_df$composite_id),
  barcode_term = "MiFishU",
  local_corroboration = local_corr,
  skip_locally_corroborated = TRUE,
  cache_dir = "ref_eval_cache" # persists across reruns
)

# 3. Combine both signals into one keep/remove verdict per accession.
ref_labeled <- score_reference_labels(ref_eval, local_corroboration = local_corr)
bad_accessions <- ref_labeled$accession[ref_labeled$reference_action == "remove"]

# 4. Exclude before training a likelihood model.
reference_df_clean <- reference_df[!reference_df$composite_id %in% bad_accessions, ]
```

On a real Great Lakes fish reference set (~2,650 accessions), this pattern resolved most accessions for free via local corroboration and BLASTed only the remainder — of the ~230 accessions that ever looked questionable

under any measure tried, only 3 (~1.3%) were confirmed genuine mislabels. Most apparent problems are same-submission-batch artifacts or tight-congener confusion a broad, independent database can correctly clear.

`score_reference_labels()`'s `reference_action` is "keep"/"caution"/"inspect"/"remove"/"untested" — "caution"/"inspect" mark accessions worth a human look without being confident enough to remove outright, and a locally-corroborated accession can **veto** an otherwise-"remove" BLAST verdict down to "inspect" (`action_reason` = "vetoed_by_local_corroboration"). See `score_reference_labels()`'s own documentation for the full decision framework, and `review_flagged_accessions()` (below) for an optional LLM second look at ambiguous cases before removing anything.

`evaluate_reference_accessions()` BLASTs each accession's own deposited sequence against a broad, unrestricted NCBI database, keeps hits that are genuinely independent (not the same submission batch), and asks whether the closest independent hits agree with the accession's own listed taxon.

Getting an accession list from your own downloaded/BLASTed sequences: `evaluate_reference_accessions()` takes NCBI accessions, not sequences directly — but you don't need to look any up by hand. `blast_sequences()` (see Quick Start above) already returns the reference database's own accession for every hit, so the accessions worth screening are just the candidates your own queries actually matched:

```
blast_hits <- blast_sequences(filtered, database = "nt", barcode_term = "12S")

qc <- evaluate_reference_accessions(
  unique(blast_hits$accession),
  cache_dir = tools::R_user_dir("TaxaMatch", "cache")
)
qc[, c("accession", "listed_taxon", "hierarchy_flag", "finest_common_rank")]
```

If instead you're starting from a locally downloaded reference FASTA (e.g. fetched via `TaxaLikely::fetch_ncbi_reference_sequences()` or downloaded by hand from NCBI) rather than your own BLAST hits, `read_sequence_table()` already extracts the accession from each header's first whitespace-delimited token (default `header_format` = "none") — no manual header parsing needed:

```
ref_seqs <- read_sequence_table("my_reference_sequences.fasta")
qc <- evaluate_reference_accessions(unique(ref_seqs$accession))
```

`evaluate_reference_accessions()` re-fetches each accession's own sequence from NCBI directly (it doesn't take a sequence as input) — this only needs the accession strings, not the downloaded sequence content itself.

`hierarchy_flag` is "congruent" / "incongruent" / "insufficient_independent_evidence". "incongruent" is not a verdict on its own, a genuine mislabel and "this marker has poor resolving power for this lineage" produce the same flag; the identity diagnostics (`best_agreeing_pident`, `best_disagreeing_pident`, `best_disagreeing_taxon`, `congruent_evidence_exists_anywhere`) distinguish them. Read `inst/reference_accession_evaluation_guide.md` before acting on a flagged accession.

Consume the result with `flag_incongruent_references()` (the recommended default — annotates a match object, never removes a row) or `remove_incongruent_references()` (a deliberate, reviewed opt-in that drops "incongruent" rows):

```
match_df <- flag_incongruent_references(match_df, qc)
```

For a deeper dive on one specific flagged accession, `investigate_flagged_accession()` (singular) / `investigate_flagged_accessions()` (batch) re-BLAST against the accession's own listed species and its top disagreeing taxon specifically, with cached results. `check_marker_mismatch()` is a cheap pre-check:

does the record's own annotated `/gene//product` qualifier actually match the marker an evaluation was scoped to (catches e.g. a 16S sequence deposited under a 12S-scoped audit).

A large accession list is resilient to NCBI rate-limiting/CPU-budget throttling by default. `evaluate_reference_accessions()` processes accessions `chunk_size` at a time (default 200L), writing the persistent cache after each chunk rather than once at the end – an interruption only loses whatever chunk was still in flight. If `blast_sequences()`'s own circuit breaker (`max_consecutive_batch_failures`, default 3L) detects sustained batch failures, `evaluate_reference_accessions()` stops itself early rather than grinding through every remaining accession at up to 30 minutes per doomed BLAST batch – everything evaluated so far stays cached, and a `message()` reports how much completed and recommends a pause before calling the exact same command again to resume (already-cached accessions are read straight from cache, not re-BLASTed):

```
qc <- evaluate_reference_accessions(large_accession_list, cache_dir = my_cache_dir)
# if NCBI throttles partway through:
#   evaluate_reference_accessions(): stopped early -- NCBI appears to be
#   rate-limiting or CPU-throttling this connection.
#   412 of 1183 accession(s) resolved this call (34.8%); 771 still pending.
#   ...
#   Recommended: wait at least 15 minutes, then call evaluate_reference_
#   accessions() again with the SAME accessions and cache_dir.
attr(qc, "run_summary") # n_total, n_evaluated_this_call, pct_complete, ...
```

LLM second-look review — `review_flagged_accessions()` sends the flagged/borderline subset (`hierarchy_flag %in% c("incongruent", "insufficient_independent_evidence")` plus non-species-resolved accessions) to an LLM for a free-text second look, the same “narrative judgment layer on top of statistical flags, never replacing them” pattern `TaxaFlag::review_assignments()` uses for posterior assignments. It adds what the statistical check can't — recognizing a known hybrid-cross name or an informal specimen code — but never re-decides `hierarchy_flag` itself. LLM calls are real, billed API cost, so this also caches: an accession already reviewed with *unchanged* inputs is served from `cache_dir` instead of a fresh call, but a genuine change (e.g. re-running `evaluate_reference_accessions()` flips `hierarchy_flag` or `best_disagreeing_taxon` for that accession) triggers a real re-review automatically — the cache is keyed on a content fingerprint of the review-relevant columns, not just the accession name:

```
qc_reviewed <- review_flagged_accessions(qc, cache_dir = my_cache_dir)
qc_reviewed[!is.na(qc_reviewed$accession_review_comment),
  c("accession", "accession_likely_explanation", "accession_review_comment",
    "accession_review_cache_hit")]

# Re-running the same call makes zero new LLM calls -- everything with
# unchanged inputs is served from cache_dir.
```

Downstream Tools

`TaxaMatch` produces a match object; `TaxaLikely` converts scores to likelihoods; `TaxaAssign` computes Bayesian posteriors. From there:

- **cameratrapp** – detection rates, activity patterns, survey design from camera trap data
- **Distance / unmarked** – occupancy and abundance modeling
- **TaxaFlag** – flags anomalous detections (lab contamination, geographic outliers)

`TaxaMatch`'s `observation_id` (image filename stem for camera traps, `{recording}-{start}-{end}` for acoustics) links posteriors back to the original media files for review.

Match Object Output

The canonical match object has one row per `observation_id` x reference match:

Column	Description
<code>observation_id</code>	Query identifier (ESV/ASV ID)
<code>score_original</code>	Raw match score (original scale, never modified)
<code>taxon_name</code>	Best taxon label at finest available rank
<code>taxon_name_rank</code>	Rank of <code>taxon_name</code> (species, genus, etc.)
<code>family, genus, species, ...</code>	Taxonomy columns

`blast_sequences()` also attaches `attr(match_obj, "report_params")` (`method`, `database`, `min_score`, `min_query_coverage`, `n_samples`). `min_query_coverage` is the alignment-coverage floor the match object was built under; `TaxaLikely::train_likelihood_model(min_pair_coverage =)` must equal it as a fraction, and `TaxaLikely::evaluate_likelihoods()` reads it from here to warn on a mismatch.

Vignettes

- Match Standardization – full workflow

Part of TaxaID

TaxaMatch standardizes match data for two downstream paths: TaxaLikely (Bayesian likelihood model) or TaxaAssign (direct LLM-based assignment).

Ecosystem: TaxaTools -> **TaxaMatch** -> TaxaLikely -> TaxaAssign

See the TaxaID README for ecosystem overview and installation instructions.

Citation

Lafferty, K.D., 2026, TaxaID – A modular R ecosystem for Bayesian taxonomic assignment: U.S. Geological Survey software release, <https://doi.org/10.5066/xxxxxx>.

Software Requirements

- R ($\geq 4.1.0$; R Core Team 2025)
- TaxaTools (foundation package, installed first)
- httr2 and rentrez (for remote NCBI BLAST)
- Biostrings and rBLAST (Hahsler and Nagar 2024; optional, for local BLAST)
- Python 3.9+ and birdnetlib (`pip3 install birdnetlib`; optional, for acoustic analysis via BirdNET-Analyzer)

All dependencies are declared in the DESCRIPTION file and installed automatically.

Developed with Claude Code (Anthropic PBC, San Francisco, California).

References

- Altschul, S.F., Gish, W., Miller, W., Myers, E.W. and Lipman, D.J. (1990). Basic local alignment search tool. *Journal of Molecular Biology*, 215(3), 403–410.
- Chiranjeevi, S., Saadati, M., Deng, Z.K., Koushik, J., Jubery, T.Z., Mueller, D.S., O’Neal, M., Merchant, N., Singh, Aarti, Singh, A.K., Sarkar, S., Singh, Arti and Ganapathysubramanian, B. (2025). InsectNet: Real-time identification of insects using an end-to-end machine learning pipeline. *PNAS Nexus*, 4(1), pgae575. <https://doi.org/10.1093/pnasnexus/pgae575>
- Hahsler, M. and Nagar, A. (2024). rBLAST: R Interface for the Basic Local Alignment Search Tool. R package version 0.99.4. Bioconductor. <https://doi.org/10.18129/B9.bioc.rBLAST>
- R Core Team (2025). R: A Language and Environment for Statistical Computing. V.4.5.2. R Foundation for Statistical Computing, Vienna, Austria. <https://www.r-project.org>
- Tabak, M.A., Norouzzadeh, M.S., Wolfson, D.W., Sweeney, S.J., Vercauteren, K.C., Snow, N.P., Halseth, J.M., Di Salvo, P.A., Lewis, J.S., White, M.D., Teton, B., Beasley, J.C., Schlichting, P.E., Boughton, R.K., Wight, B., Newkirk, E.S., Ivan, J.S., Odell, E.A., Brook, R.K., Lukacs, P.M., Moeller, A.K., Mandeville, E.G., Clune, J. and Miller, R.S. (2019). Machine learning to classify animal species in camera trap images: applications in ecology. *Methods in Ecology and Evolution*, 10(4), 585–590.